```
import re
from PySide6.QtCore import QObject, Qt, Slot, QModelIndex
from PySide6.QtGui import QStandardItemModel, QStandardItem

from IndexTreeView import CaseInsensitiveItem  # Import your custom class
from EditorTab import EditorTab

# --- Explicit Architectural Data Roles Synchronization ---
ROLE_PATH = Qt.UserRole          # File path string or raw tag name
ROLE_LINE = Qt.UserRole + 1      # Line number (int)
ROLE_IS_LEAF = Qt.UserRole + 2   # Boolean leaf node flag (True = file, False = folder)
ROLE_COL = Qt.UserRole + 3       # Column number (int)
ROLE_COUNT = Qt.UserRole + 4     # Occurrences count (int) for folders

# Custom Roles to hold the UID structure on the IDn cells
# ROLE_UID_DATA = Qt.UserRole      # Dictionary: {"id": int, "path": str, "line": int, "col": int}

class IndexTreeController(QObject):
    def __init__(self, tree_view=None, editor_tab=None, parent=None):
        # Pass parent up to maintain QObject memory safety tree
        super().__init__(parent)

        # 1. Store view components safely
        self.tree_view = tree_view
        self.editor_tab = editor_tab

        # 2. Initialize internal data representation properties
        self.model = QStandardItemModel(self)
        self.model.setHorizontalHeaderLabels(["Index Terms", "References"])
        self.ROLE_UID_DATA = Qt.ItemDataRole.UserRole + 1

        # 3. Setup signal handling logic safely if views are provided
        if self.tree_view:
            self.tree_view.setModel(self.model)
            self.tree_view.clicked.connect(self.on_tree_item_clicked)

    # def __init__(self, parent=None):
    #     super().__init__(parent)
    #     self.model = QStandardItemModel(self)
    #     self.model.setHorizontalHeaderLabels(["Index Terms", "References"])
    #     self.ROLE_UID_DATA = Qt.ItemDataRole.UserRole + 1

    @Slot(QModelIndex)
    def on_tree_item_clicked(self, model_index):
        """
        Fires whenever an item in the 2-column QTreeView is clicked.
        Extracts metadata safely and routes execution to the editor tab.
        """
        if not model_index.isValid() or not self.editor_tab:
            return

        # Safely extract your metadata payload dictionary from the model
        uid_dict = model_index.data(self.ROLE_UID_DATA)
        if not uid_dict or not isinstance(uid_dict, dict):
            return

        # Extract coordinates generated by LatexIndexParser
        target_line = uid_dict.get("line_number")
        target_col = uid_dict.get("column_offset")

        if target_line is not None and target_col is not None:
            # Route target jumps cleanly to your custom subclassed QPlainTextEdit
            self.editor_tab.jump_to_coordinates(
                line=target_line,
                column=target_col,
                is_one_indexed=True
            )

    @Slot(list, list)
    def populate_from_worker_payloads(self, headings: list, references: list):
        """Processes background thread payload matrices safely into the model structure."""
        self.model.layoutAboutToBeChanged.emit()
        self.model.removeRows(0, self.model.rowCount())
        self.model.setHorizontalHeaderLabels(["Index Terms", "References"])

        # Comprehensive normalizer that strips \string, vertical bars, and encapsulations
        def normalize_raw_path(raw_path_str: str) -> str:
            if not raw_path_str:
                return ""
            # Clean string control tokens and strip trailing page modifiers
            txt = raw_path_str.replace(r'\string', '').strip()
            if '|' in txt:
                txt = txt.split('|')[0].strip()
            # Normalize slashes to exclamation marks
            txt = txt.replace("/", "!")
            return "!".join([p.strip() for p in txt.split("!") if p.strip()])
```

```
    id_to_refs = {}
    path_to_refs = {}

    safe_references = references if references is not None else []
    for ref in safe_references:
        if not ref:
            continue
        h_id = ref.get("heading_id") or ref.get("id")
        if h_id is not None:
            id_to_refs.setdefault(int(h_id), []).append(ref)

        raw_path = ref.get("heading_raw_text") or ref.get("heading_text") or ref.get("name") or ""
        if raw_path:
            clean_path = normalize_raw_path(raw_path)
            path_to_refs.setdefault(clean_path.lower(), []).append(ref)

    safe_headings = headings if headings is not None else []
    for head in safe_headings:
        if not head:
            continue
        heading_raw_text = head.get("heading_text") or head.get("name") or ""
        if not heading_raw_text:
            continue

        # Compute a clean structural path for tree branching
        clean_heading = normalize_raw_path(heading_raw_text)
        parts = [p.strip() for p in clean_heading.split("!") if p.strip()]
        if not parts:
            continue

        h_id = head.get("id")
        associated_refs = []
        if h_id is not None and int(h_id) in id_to_refs:
            associated_refs = id_to_refs[int(h_id)]
        else:
            current_full_path = clean_heading.lower()
            associated_refs = path_to_refs.get(current_full_path, [])

        self._insert_or_merge_hierarchical_node(self.model.invisibleRootItem(), parts, associated_refs)

    self.model.sort(0, Qt.SortOrder.AscendingOrder)
    self.model.layoutChanged.emit()

def _insert_or_merge_hierarchical_node(self, parent_item, remaining_parts: list, refs: list):
    """Recursively compiles hierarchical structures, applying tokens exclusively to leaves."""
    if not remaining_parts:
        return

    current_token = remaining_parts[0]
    match_found = None

    for row in range(parent_item.rowCount()):
        child_col0 = parent_item.child(row, 0)
        if child_col0 and child_col0.text().strip().lower() == current_token.strip().lower():
            match_found = child_col0
            break

    # if match_found:
    #     target_branch = match_found
    # else:
    #     # Column 0 manages the text node branch case-insensitively
    #     branch_item = CaseInsensitiveItem(current_token)
    #     branch_item.setEditable(False)

    #     # Column 1 serves as the structural reference token holder
    #     ref_item = QStandardItem("")
    #     ref_item.setEditable(False)

    #     # CRITICAL FLAG MANAGEMENT: Column 0 can be highlighted; Column 1 rejects selection highlights
    #     branch_item.setFlags(branch_item.flags() | Qt.ItemFlag.ItemIsSelectable | Qt.ItemFlag.ItemIsEnabled)
    #     ref_item.setFlags((ref_item.flags() & ~Qt.ItemFlag.ItemIsSelectable) | Qt.ItemFlag.ItemIsEnabled)

    #     parent_item.appendRow([branch_item, ref_item])
    #     target_branch = branch_item

    if match_found:
        target_branch = match_found
    else:
        # Column 0 manages the text node branch case-insensitively via our upgraded class
        # Passing 'apple@\textit{apple}' or '\textit{Zebra}' preserves formatting tokens
        # for the Item Delegate, while sorting purely by alphabetic keys
        branch_item = CaseInsensitiveItem(current_token)
        branch_item.setEditable(False)

        # Column 1 serves as the structural reference token holder
        ref_item = QStandardItem("")
```

```
            ref_item.setEditable(False)

            # Assign matching layout view flags
            branch_item.setFlags(branch_item.flags() | Qt.ItemFlag.ItemIsSelectable | Qt.ItemFlag.ItemIsEnabled)
            ref_item.setFlags((ref_item.flags() & ~Qt.ItemFlag.ItemIsSelectable) | Qt.ItemFlag.ItemIsEnabled)

            parent_item.appendRow([branch_item, ref_item])
            target_branch = branch_item

        if len(remaining_parts) == 1:
            row_idx = target_branch.row()
            actual_parent = target_branch.parent() or self.model.invisibleRootItem()
            sibling_ref_item = actual_parent.child(row_idx, 1)

            existing_records = sibling_ref_item.data(self.ROLE_UID_DATA) or []
            new_records = list(existing_records) if isinstance(existing_records, list) else []

            safe_refs = refs if refs is not None else []
            for r in safe_refs:
                if not r:
                    continue
                r_uid = r.get("uid") or f"{r.get('file_path')}:{r.get('line_number')}:{r.get('column_offset')}"
                if r_uid not in [ex.get("uid") for ex in new_records if ex]:
                    new_records.append({
                        "uid": r_uid,
                        "unique_id_number": int(r.get("unique_id_number") or r.get("id") or 0),
                        "file_path": str(r.get("file_path") or ""),
                        "line_number": int(r.get("line_number") or r.get("line") or 0),
                        "column_offset": int(r.get("column_offset") or r.get("col") or 0),
                        "encap": r.get("encap", "standard")
                    })

            sibling_ref_item.setData(new_records, self.ROLE_UID_DATA)

            unique_ids = sorted(list(set(rec["unique_id_number"] for rec in new_records if rec.get("unique_id_number"))))
            if unique_ids:
                sibling_ref_item.setText(" ".join(f"[{uid}]" for uid in unique_ids))
            else:
                sibling_ref_item.setText("")
        else:
            row_idx = target_branch.row()
            actual_parent = target_branch.parent() or self.model.invisibleRootItem()
            sibling_ref_item = actual_parent.child(row_idx, 1)
            if sibling_ref_item and not sibling_ref_item.data(self.ROLE_UID_DATA):
                sibling_ref_item.setText("")

            self._insert_or_merge_hierarchical_node(target_branch, remaining_parts[1:], refs)

# class IndexTreeController:
#     def __init__(self, tree_view: QTreeView):
#         self.tree_view = tree_view
#         # Bind cleanly onto the view's internal base model instance
#         self.model = self.tree_view.base_model

#     def populate_from_worker_payloads(self, headings: list, references: list):
#         """
#         Safely processes data matrices across thread boundaries.
#         Supports both transient file-tree scrapes and persistent database reloads.
#         """
#         self.model.layoutAboutToBeChanged.emit()

#         # Reset the view model canvas cleanly without dropping column configurations
#         self.model.removeRows(0, self.model.rowCount())
#         self.model.setHorizontalHeaderLabels(["Index Terms", "References"])

#         # Map references by both ID and clean text path to cover both live scrapes and db reloads
#         id_to_refs = {}
#         path_to_refs = {}

#         safe_references = references if references is not None else []
#         for ref in safe_references:
#             if not ref:
#                 continue

#             # Track by ID for database loads
#             h_id = ref.get("heading_id") or ref.get("id")
#             if h_id is not None:
#                 id_to_refs.setdefault(int(h_id), []).append(ref)

#             # Track by clean context path string for live file-tree scrapes
#             raw_path = ref.get("heading_raw_text") or ref.get("heading_text") or ref.get("name")
#             if raw_path:
#                 # Standardize path structure cleanly
#                 clean_path = "!".join([p.strip() for p in raw_path.replace("/", "!").split("!") if p.strip()])
#                 path_to_refs.setdefault(clean_path.lower(), []).append(ref)

#         # Process the heading nodes into the tree model
```

```
#         safe_headings = headings if headings is not None else []
#         for head in safe_headings:
#             if not head:
#                 continue

#             heading_raw_text = head.get("heading_text") or head.get("name") or ""
#             if not heading_raw_text:
#                 continue

#             # Uniform segment formatting: converts 'Animals/Mammals' or 'Animals!Mammals' to ['Animals', 'Mammals']
#             normalized_text = heading_raw_text.replace("/", "!")
#             parts = [p.strip() for p in normalized_text.split("!") if p.strip()]
#             if not parts:
#                 continue

#             # Resolve references by matching either the DB id or the unified string path
#             h_id = head.get("id")
#             associated_refs = []

#             if h_id is not None and int(h_id) in id_to_refs:
#                 associated_refs = id_to_refs[int(h_id)]
#             else:
#                 # Fallback for live scraping paths where database entry records do not exist yet
#                 current_full_path = "!".join(parts).lower()
#                 associated_refs = path_to_refs.get(current_full_path, [])

#             # Inject node recursively into tree context
#             self._insert_or_merge_hierarchical_node(self.model.invisibleRootItem(), parts, associated_refs)

#         # Alphabetize view nodes via standard model rules
#         self.model.sort(0, Qt.SortOrder.AscendingOrder)
#         self.model.layoutChanged.emit()

#     def _insert_or_merge_hierarchical_node(self, parent_item, remaining_parts: list, refs: list):
#         """Recursively builds subheading trees, applying unique reference tokens to leaf nodes."""
#         if not remaining_parts:
#             return

#         current_token = remaining_parts[0]
#         match_found = None

#         # Case-insensitive structural sibling lookup
#         for row in range(parent_item.rowCount()):
#             child_col0 = parent_item.child(row, 0)
#             if child_col0 and child_col0.text().strip().lower() == current_token.strip().lower():
#                 match_found = child_col0
#                 break

#         if match_found:
#             target_branch = match_found
#         else:
#             # Column 0 manages hierarchical heading branches using CaseInsensitiveItem
#             branch_item = CaseInsensitiveItem(current_token)
#             ref_item = QStandardItem("")

#             branch_item.setEditable(False)
#             ref_item.setEditable(False)
#             # CRITICAL FIX 1: Column 0 stays selectable for context menus; Column 1 blocks selection highlights
#             branch_item.setFlags(branch_item.flags() | Qt.ItemFlag.ItemIsSelectable | Qt.ItemFlag.ItemIsEnabled)
#             ref_item.setFlags((ref_item.flags() & ~Qt.ItemFlag.ItemIsSelectable) | Qt.ItemFlag.ItemIsEnabled)

#             parent_item.appendRow([branch_item, ref_item])
#             target_branch = branch_item

#         # Check if we have arrived at the leaf node of this path array
#         if len(remaining_parts) == 1:
#             row_idx = target_branch.row()
#             actual_parent = target_branch.parent() or self.model.invisibleRootItem()
#             sibling_ref_item = actual_parent.child(row_idx, 1)

#             # Fetch existing records embedded in the structural tracking roles
#             existing_records = sibling_ref_item.data(Qt.UserRole + 1) or []
#             new_records = list(existing_records) if isinstance(existing_records, list) else []

#             # Append reference instances safely, keeping complete tracking data payloads intact
#             safe_refs = refs if refs is not None else []
#             for r in safe_refs:
#                 if not r:
#                     continue

#                 # Check for structural identity using geometry hashes to merge duplicates
#                 r_uid = r.get("uid") or f"{r.get('file_path')}:{r.get('line_number')}:{r.get('column_offset')}"
#                 if r_uid not in [ex.get("uid") for ex in new_records if ex]:
#                     # Format layout naming parameters safely to mirror spec definitions
#                     clean_rec = {
#                         "uid": r_uid,
#                         "unique_id_number": int(r.get("unique_id_number") or r.get("id") or 0),
```

```
#                         "file_path": str(r.get("file_path") or ""),
#                         "line_number": int(r.get("line_number") or r.get("line") or 0),
#                         "column_offset": int(r.get("column_offset") or r.get("col") or 0),
#                         "encap": r.get("encap", "standard")
#                     }
#                     new_records.append(clean_rec)

#             # Save structural tracking configurations into UserRole data mapping
#             sibling_ref_item.setData(new_records, Qt.UserRole + 1)

#             # Extract and sort exclusively by unique session identifier values
#             unique_ids = sorted(list(set(rec["unique_id_number"] for rec in new_records if rec.get("unique_id_number"))))

#             if unique_ids:
#                 # Render formatted token strings matching your layout spec: "[1] [12] [44]"
#                 token_str = " ".join(f"[{uid}]" for uid in unique_ids)
#                 sibling_ref_item.setText(token_str)
#             else:
#                 sibling_ref_item.setText("")
#         else:
#             # Clear text leaking into upper parent structural branches
#             row_idx = target_branch.row()
#             actual_parent = target_branch.parent() or self.model.invisibleRootItem()
#             sibling_ref_item = actual_parent.child(row_idx, 1)
#             if sibling_ref_item and not sibling_ref_item.data(Qt.UserRole + 1):
#                 sibling_ref_item.setText("")

#             # Advance down the array to build nested structures
#             self._insert_or_merge_hierarchical_node(target_branch, remaining_parts[1:], refs)
```